

Personal Reminder App

Complete PRD + TRD + UI/UX + Architecture + Development Plan

Product concept

A beautiful personal reminder assistant that remembers tasks, birthdays, important dates and recurring activities, and alerts the user exactly when they want.

The app should work **offline-first** for reminders. Internet should not be required for a scheduled reminder to fire.

1. Design Direction

Your reference image establishes the visual direction:

- Purple/lavender primary environment
- Soft gradient backgrounds
- Large rounded containers
- Layered cards
- Floating surfaces
- Illustration/visual hero areas
- Pill-shaped controls
- Soft shadows
- White/near-white foreground surfaces
- Rounded avatars
- Compact typography
- Large visual hierarchy

I would evolve it into:

"Soft Glass + Modern Skeuomorphism"

Not extreme glassmorphism everywhere.

Instead:

Background



Gradient / ambient blobs



Glass surface



Elevated card



Interactive pill



Native-looking controls

This will make it look premium without becoming visually noisy.

2. Design System

Primary visual palette

Light

Background:

#F4F1FF

Primary:

#6658E8

Primary Dark:

#5144C9

Secondary:

#A77AF3

Accent:

#7BC99A

Surface:

rgba(255,255,255,0.72)

Card:

#FFFFFF

Text:

#17152B

Secondary Text:

#77758A

Dark

Background:

#11101B

Surface:

rgba(37,35,54,0.75)

Card:

#211F30

Primary:

#8B7CFF

Text:

#F7F5FF

Secondary:

#AAA6BD

3. Typography

Use a modern rounded typeface.

Recommended:

Inter / SF Pro / Manrope

For the app's visual personality, I'd use:

Headings

Manrope

600 / 700

Body

Inter

400 / 500

Numbers

Use slightly heavier weight for:

- Time
 - Countdown
 - Task progress
 - Dates
-

4. Font Size Philosophy

You specifically requested **small fonts**.

I agree, but don't make important information too small.

Recommended:

Element	Size
Hero title	26–32

Element	Size
Screen title	22–26
Card title	15–17
Body	13–15
Metadata	11–12
Caption	10–11
Time	16–22
Large countdown	32–42

Use **small typography + strong hierarchy**, rather than simply shrinking everything.

5. Corner Radius System

Use consistent radius tokens.

XS 8px

SM 12px

MD 16px

LG 22px

XL 28px

PILL 999px

Main cards:

20–28px

Buttons:

14–18px

Chips:

999px

6. Glassmorphism Rules

Don't make every component transparent.

Glass Card

Background:

`rgba(255,255,255,0.60)`

Blur:

20–30px

Border:

1px `rgba(255,255,255,0.55)`

Shadow:

0 12px 35px `rgba(...)`

Use glass for:

- Floating dashboard cards
- Bottom sheets
- Filters
- Quick actions
- Calendar overlays
- Floating navigation

Use solid surfaces for:

- Forms
- Important confirmation dialogs
- Notification content
- Accessibility-critical elements

7. Skeuomorphism Usage

Use **micro-skeuomorphism**, not old-style realistic buttons.

Examples:

Alarm switch

A slightly physical-looking toggle.

Time selector

Raised circular time controls.

Completed task

Soft inset/check effect.

Calendar date

Selected date appears physically raised.

Important reminder

Subtle glowing elevation.

This gives the application a tactile feel.

8. Pill UI

Pills should be heavily used where appropriate.

Examples:

[Today]

[Upcoming]

[Birthday]

[High Priority]

[Work]

[Daily]

Segmented controls:

[Day] [Week] [Month]

Category selection:

[Personal] [Work] [Family]

But don't make every button a pill. Otherwise the UI becomes repetitive.

9. App Navigation

I recommend:

Home

Tasks

Calendar

People

More

with a central floating +.

Visual structure:

Content

Home Tasks + Calendar People

The + should be elevated slightly above the navigation bar.

10. Splash Screen

Minimal.



REMINDLY

Remember what matters.

Animation:

Logo



Soft glow



Fade into Home

11. Onboarding

Only 3 screens.

Screen 1

Remember everything.

Illustration:

Calendar + bell + task cards.

Your personal reminder
assistant.

Never forget an important
moment again.

Screen 2

Remind me my way.

Show:

Daily

Weekly

Custom

Before event

Birthday

Screen 3

Stay on top of life.

Show:

Tasks

Birthdays

Appointments

Important dates

CTA:

Get Started

12. Authentication

I recommend **not forcing account creation immediately**.

The app can support:

Continue as Guest

and:

Sign in with Google

Sign in with Apple

Email

This is also aligned with Apple's guidance to avoid unnecessarily forcing sign-in before users can experience the app. ([Apple Developer](#))

13. Google Login Flow

Your requested flow:

Login Screen

↓

Continue with Google

↓

Native Google authentication UI

↓

User chooses Google account

↓

Google authentication

↓

Backend verifies identity

↓

Existing account?

/\

YES NO

↓ ↓

Home Create profile

Important: don't create your own fake Google account-selection popup.

Use the platform's supported Google authentication flow.

For a Flutter implementation, the current official `google_sign_in` package supports Android and iOS; its current API has separate authentication and authorization concepts. ([Dart packages](#))

For Android specifically, Google's current guidance says the older Google Sign-In API is outdated and recommends **Credential Manager** for modern Android authentication. ([Google for Developers](#))

So I would architect the authentication layer so that the UI isn't tightly coupled to one authentication SDK.

14. Apple Login

For iOS:

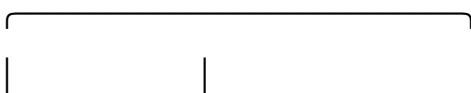
Continue with Apple

should also be available.

This is especially important if the app uses Google/social login as an account-authentication method; Apple's current App Store guideline 4.8 requires an equivalent privacy-preserving login option in covered cases. ([Apple Developer](#))

Apple's Sign in with Apple guidance also supports private relay email addresses, so the backend must not assume the user's Apple email equals their normal email. ([Apple Developer](#))

15. Login Screen UI



	◆	
	Rememberly	
	Your life, remembered.	
	G Continue Google	
	🍏 Continue with Apple	
	— or —	
	Continue with Email	
	Continue as Guest	

The Google/Apple buttons should retain recognizable platform branding rather than being excessively stylized. Apple's guidance specifically recommends using recognizable/system-provided Sign in with Apple buttons and keeping them prominent. ([Apple Developer](#))

16. First Login Profile

After Google login:

Welcome, Akash 🙌

Let's personalize your reminders.

Your name

[Akash]

Profile photo

[Google photo]

Default reminder time

[9:00 PM]

Timezone

[Asia/Kolkata]

[Continue]

Don't ask 15 questions.

Collect only what is needed.

17. Permission Onboarding

After login:

Make reminders reliable

 Notifications

Required for reminders.

 Exact alarms

Used for precise reminder timing.

Battery settings

Recommended on Android.

[Enable Notifications]

Explain **why** each permission is needed.

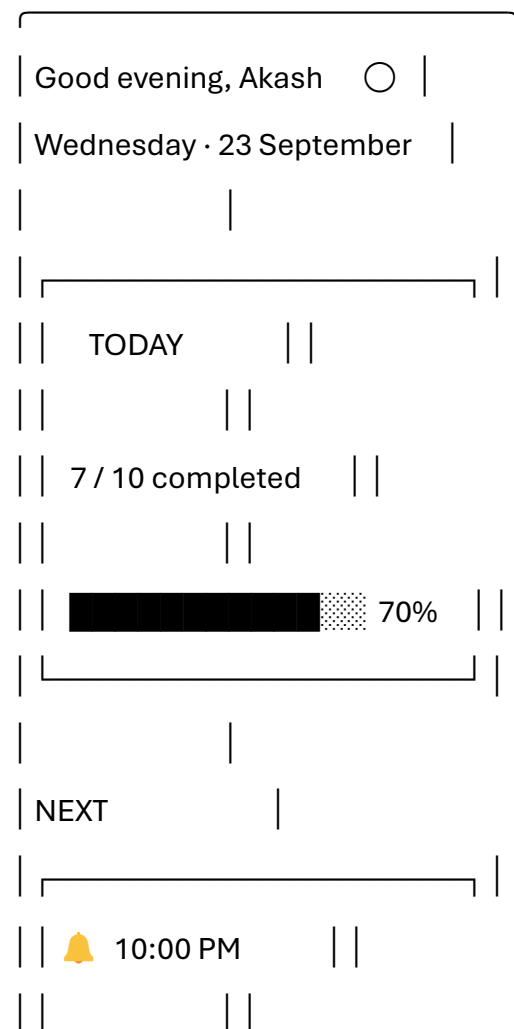
Never simply show:

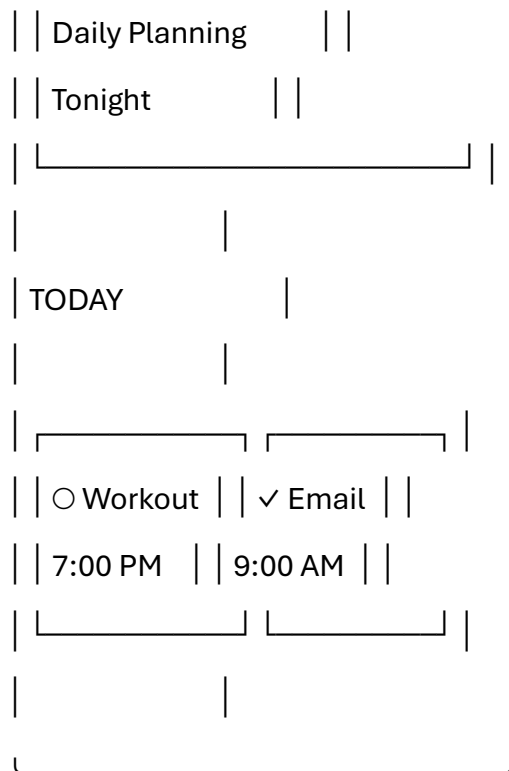
"Allow everything."

18. Home UI

This should be the app's signature screen.

Inspired by your reference:





19. Home Hero Area

Borrow the visual storytelling from your reference.

Instead of medicine illustrations:

Reminder illustration

A person surrounded by:

- calendar
- bell
- checklist
- birthday balloon
- clock
- stars

The illustration should occupy approximately **25–35% of the hero area**, not dominate the entire screen.

20. Today Date Selector

Reference-inspired horizontal date selector:

September 2026

2122232425

MonTueWedThuFri

23

Wed

Selected date:

- raised
- glass surface
- primary accent
- subtle shadow

21. Today's Task Cards

Use compact cards.

Client Proposal

Send final PDF

8:00 PM [Work]

Swipe actions:

Swipe right → Complete

Swipe left → Snooze/Delete

22. Add Reminder

Large bottom sheet:

New Reminder

What do you need to remember?

[Finish proposal...]

[Today] [Tomorrow]

[8:00 PM]

[ Reminder]

More options ›

[Create Reminder]

This is the **quick-add mode**.

23. Advanced Reminder Screen

When user taps **More options**:

Create Reminder

Title

Description

WHEN

Start date

Start time

End date

End time

REPEAT

Never

Daily

Weekly

Monthly

Yearly

Custom

ALERTS

At time

Before event

Custom alerts

ALARM

Sound

Vibration

Snooze

ORGANIZE

Category

Priority

Tags

[Save Reminder]

24. Multiple Alerts

Very important.

One reminder can have multiple alerts.

Example:

Father's Birthday

Alerts

✓ Oct 25 · 10:00 PM

✓ Oct 26 · 10:00 PM

✓ Oct 27 · 10:00 PM

✓ Oct 28 · 10:00 PM

✓ Oct 29 · 10:00 PM

✓ Oct 30 · 09:00 AM

But internally don't necessarily create six independent user reminders.

Store a **recurrence/alert rule**.

25. Birthday UI

The People tab should feel different from Tasks.

People



Birthdays

	
Dad	
Father	
30 October	
37 days	
[View Reminder]	

26. Birthday Add Screen

Add Birthday



Add Photo

Name

[Father]

Relationship

[Father ▼]

Birthday

[30 October]

REMIND ME

Start

[5 days before]

Time

[10:00 PM]

Repeat

[Every day]

Stop

[Birthday]

☒ Birthday-day notification

[Save Birthday]

This directly supports your father's birthday scenario.

27. Birthday Countdown Card

On Home:

🎂	FAMILY
	Dad's Birthday
	37 DAYS
	30 October
Reminding daily at 10 PM	

28. Birthday Day UI

On the actual day, the design can transform:

🎉	
	TODAY!
	Dad's Birthday
	30 October
	[Call] [WhatsApp]
	[Birthday Message]

29. Calendar UI

Use a modern floating calendar.

September 2026

< >

M T W T F S S

1 2 3 4 5 6

7 8 9 10 11 12 13

14 15 16 17 18 19 20

21 22 [23] 24 25 26 27

28 29 30

TODAY

8:00 PM

Client Proposal

10:00 PM

Daily Planning

 30 Oct

Dad's Birthday

30. Tasks Screen

Top segmented navigation:

[Today] [Upcoming] [Completed]

Then:

TODAY

Morning

☐ Exercise

Afternoon

☐ Call client

Evening

☐ Review proposal

☐ Daily planning

31. Swipe Gestures

Modern mobile UX:

Swipe right

Complete.

Swipe left

Actions:

Snooze

Edit

Delete

Long press

Context menu.

Tap

Open details.

Drag

Reorder.

32. Reminder Detail

Reminder



Daily Planning

Review tomorrow's priorities
and prepare task list.

TODAY

10:00 PM

Repeat

Every day

Until

No end date

Sound

Default

Vibration

On

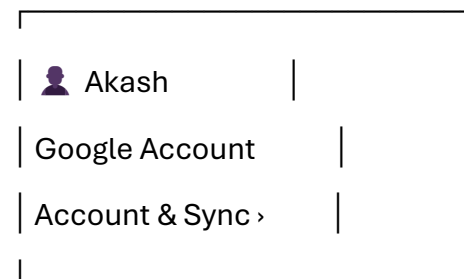
Snooze

10 minutes

[Edit Reminder]

33. More Tab

More



Templates

Statistics

Notification History

Appearance

Notifications

Sound & Vibration

Widgets

Categories

Backup & Sync

Privacy

About

34. Widget System

I would include widgets in the architecture from the beginning.

Small

NEXT REMINDER

10:00 PM

Daily Planning

Medium

TODAY

☐ Proposal

☐ Workout

☒ Email

☐ Planning

Birthday

 DAD

30 OCT

37 DAYS

35. Notification Design

Notification should be useful, not generic.

Example:

Daily task

 Daily Planning

Review tomorrow's tasks.

Actions:

DONE SNOOZE

Birthday:

 Dad's birthday is tomorrow.

Actions:

VIEW MESSAGE

36. Notification Interaction

When user taps notification:

Notification



Deep Link



Reminder Detail

When tapping:

DONE

the app marks that occurrence completed.

When tapping:

SNOOZE

it schedules a new local notification.

37. PRD — Product Requirements

Functional Requirements

Authentication

- Guest mode
- Google
- Apple
- Email
- Logout
- Account deletion
- Account linking

Reminder

- Create
- Edit
- Delete
- Duplicate
- Complete
- Snooze
- Skip
- Archive
- Search
- Filter

Recurrence

- Once
- Daily
- Weekdays

- Weekly
- Monthly
- Yearly
- Custom interval
- End date
- Never-ending

Alerts

- At event
- Before event
- Multiple alerts
- Custom date/time
- Alarm sound
- Vibration
- Snooze

Birthday

- Person
- Birthday
- Relationship
- Photo
- Reminder start
- Reminder frequency
- Reminder time
- Birthday-day alert

Calendar

- Month
- Week
- Day
- Agenda

Organization

- Categories
 - Tags
 - Priority
-

38. Non-Functional Requirements

The application should target:

Performance

- App launch: ideally <2 seconds on modern devices
- Smooth 60fps UI
- Minimal unnecessary network requests

Reliability

Reminder scheduling must survive:

- Device restart
- App restart
- Timezone change
- Date change
- App update
- Permission changes
- Temporary network outage

Offline

Core reminders:

100% local-first.

39. TRD — Technical Architecture

I recommend:

Mobile

Flutter

Flutter

Dart

Material 3 foundation

Custom Design System

Flutter lets you maintain Android/iOS from one codebase while still exposing native platform capabilities where required.

The Google Sign-In Flutter plugin currently supports Android and iOS, but authentication should be abstracted behind your own AuthRepository so Google/Apple/native platform changes don't spread through the application. ([Dart packages](#))

40. Backend

Recommended:

Supabase

Supabase

└─ PostgreSQL

└─ Authentication

└─ Row Level Security

└─ Storage

└─ Edge Functions

Why?

Your application primarily needs:

- user accounts
- reminders
- people
- birthdays
- sync
- backup
- profile data

PostgreSQL is a good fit for this relational data.

41. Local Database

Use:

Drift / SQLite

Architecture:

UI

↓

ViewModel / State

↓

Use Case

↓

Repository

↓

Local DB

Cloud:

Repository

↓

Sync Service

↓

Supabase

42. Clean Architecture

Recommended project structure:

lib/

core/

└─ theme/

└─ routing/

└─ constants/

└─ errors/

└─ utils/

└─ platform/

features/

auth/

└─ data/

└─ domain/

└─ presentation/

home/

└─ data/

└─ domain/

└─ presentation/

reminders/

└─ data/

└─ domain/

└─ presentation/

tasks/

calendar/

people/

birthdays/

notifications/

settings/

sync/

43. State Management

Use one predictable state-management system.

I would use:

Riverpod

instead of mixing:

setState

Provider

Bloc

GetX

throughout the project.

44. Reminder Engine

Core architecture:

Reminder

↓

Recurrence Rule

↓

Occurrence Calculator



Next Occurrence



Local Scheduler



OS Notification

Example:

Birthday:

30 Oct

Reminder:

5 days before

Time:

10 PM

Frequency:

Daily

Engine calculates:

25 Oct 10 PM

26 Oct 10 PM

27 Oct 10 PM

28 Oct 10 PM

29 Oct 10 PM

30 Oct birthday event

45. Critical Rule

Do **not** depend on the backend for firing reminders.

Bad:

Server

↓

Cron

↓

Push

↓

Phone

Correct:

Phone

↓

Local reminder database

↓

Native OS scheduler

↓

Alarm/notification

Cloud:

Backup + synchronization

46. Native Platform Layer

Flutter should communicate with:

Android

- AlarmManager / appropriate exact scheduling APIs
- Notification channels
- Boot receiver
- Notification actions

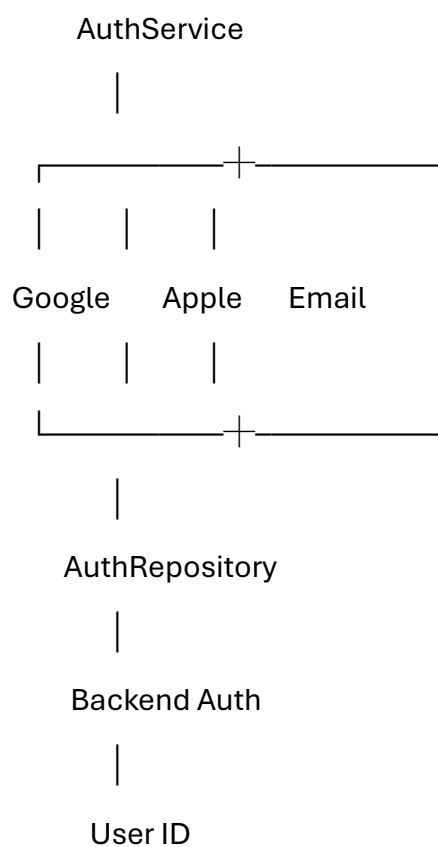
- Battery optimization guidance
- Widgets
- Credential Manager for modern Google authentication where applicable

iOS

- UserNotifications
- Notification categories/actions
- Background capabilities where appropriate
- WidgetKit
- Sign in with Apple

The native layer should be isolated behind platform interfaces.

47. Authentication Architecture



Never use:

Google email = primary database key

Use:

internal UUID

and maintain provider identities separately.

48. Database

users

id

email

display_name

avatar_url

timezone

locale

created_at

updated_at

auth_identities

id

user_id

provider

provider_subject

created_at

Providers:

google

apple

email

49. reminders

id

user_id

title

description
category_id
priority
start_at
end_at
timezone
recurrence_rule
status
created_at
updated_at

50. reminder_alerts

id
reminder_id
offset_value
offset_unit
alert_time
sound
vibration
snooze_enabled

51. people

id
user_id
name
relationship
photo_url
phone

email

notes

created_at

52. birthdays

id

person_id

birth_date

enabled

reminder_start_days

reminder_time

reminder_frequency

birthday_day_alert

This separation allows:

Person

↓

Birthday

↓

Reminder rules

instead of duplicating person information.

53. Sync Engine

Use:

Local First

↓

Change Tracking

↓

Sync Queue

↓

Server

Each record should have:

created_at

updated_at

deleted_at

sync_version

For deletes, use soft-delete/tombstones long enough for synchronization.

54. Conflict Handling

Example:

Phone:

Reminder time = 8 PM

Tablet:

Reminder time = 9 PM

Both modified offline.

Use deterministic conflict rules, preferably:

updated_at

+

server revision

But for important records, preserve conflict information rather than silently destroying user data.

55. Notification Scheduler

Create a dedicated service:

ReminderScheduler

Responsibilities:

schedule()

reschedule()
cancel()
cancelReminder()
scheduleNextOccurrence()
rebuildSchedules()
handleTimezoneChange()
handleDeviceReboot()

56. Schedule Strategy

Don't schedule 10,000 notifications.

Maintain:

Current occurrence

Next occurrence

Schedule a safe window of upcoming occurrences where platform limits make this useful.

When one fires:

Notification fires

↓

Mark occurrence delivered

↓

Calculate next occurrence

↓

Schedule next

57. App Lifecycle

First install

Install

↓

Onboarding

↓

Permission

↓

Create local DB

↓

Scheduler initialized

App launch

Load DB

↓

Check scheduler state

↓

Repair missing schedules

↓

Sync if logged in

↓

Show Home

Device reboot

BOOT

↓

Load active reminders

↓

Calculate next occurrence

↓

Reschedule

58. Notification Permission Failure

If notifications are disabled:

Reminder created ✓

But notifications are disabled.

[Enable Notifications]

Don't silently pretend the reminder will work.

59. Timezone Changes

Example:

User timezone:

Asia/Kolkata

Travels to:

Europe/London

Settings:

Reminder timezone

☒ Follow device

☐ Keep original timezone

For birthday date-only events:

30 October

should remain date-based.

60. UI Component Library

Build reusable components first.

GlassCard

GlassBottomSheet

GlassButton

PillButton
PillChip
DatePill
TimePill
ReminderCard
TaskCard
BirthdayCard
PersonCard
ProgressCard
SectionHeader
Avatar
FloatingAddButton
GlassTabBar
CalendarCell
PriorityBadge
EmptyState

This dramatically reduces inconsistent UI.

61. Design Tokens

Create:

AppColors
AppTypography
AppSpacing
AppRadius
AppShadows
AppBlur
AppMotion

Example:

spacing.xs = 4

spacing.sm = 8

spacing.md = 12

spacing.lg = 16

spacing.xl = 24

spacing.xxl = 32

Don't hard-code random values throughout the application.

62. Animation System

Keep animations subtle.

Page transition

200–300ms

Bottom sheet

250–350ms

Task completion

Checkmark + card compression

Reminder completion

Small haptic + fade

Birthday

Subtle confetti on actual birthday.

Avoid excessive animations because this is a productivity application.

63. Empty States

Example:



Nothing planned

Your day is clear.

[+ Add Reminder]

Birthday:



No birthdays yet.

Add people you never
want to forget.

[Add Birthday]

64. Error States

Don't show technical errors.

Bad:

SQLite exception 14.

Good:

We couldn't save this reminder. Please try again.

Developer logs contain the actual error.

65. Accessibility

Must support:

- Dynamic text sizing
- Screen readers
- Semantic labels
- High contrast
- Touch targets

- Reduced motion
- VoiceOver
- TalkBack

Even with your small-font aesthetic, important controls must remain accessible.

66. Security

Authentication

- OAuth/OIDC
- Secure token storage
- Refresh token handling
- Provider identity verification

Local

Use secure storage for:

- auth tokens
- sensitive configuration

Database data should be protected appropriately.

Server

Supabase RLS:

`user_id = authenticated_user_id`

A user must never be able to query another user's reminders.

67. Google Account Linking

Important edge case.

Suppose user starts as:

Guest

creates:

50 reminders

Then signs in with Google.

Do:

Guest Data

↓

Google Account

↓

Merge/Attach local data

Don't delete the existing reminders.

68. Guest Mode Architecture

Generate:

local_user_id

When guest signs in:

local_user_id

↓

authenticated_user_id

↓

migration

↓

cloud backup

This makes onboarding much smoother.

69. Analytics

Keep analytics minimal.

Track product events such as:

app_opened

reminder_created

reminder_completed

reminder_snoozed

birthday_created

notification_opened

google_signin_started

google_signin_completed

Do not collect reminder content unnecessarily.

70. Development Phases

Phase 0 — Design System

Build:

- Colors
 - Typography
 - Glass system
 - Components
 - Animations
 - Icons
 - Light/dark mode
-

Phase 1 — App Foundation

- Flutter setup
 - Routing
 - State management
 - Local DB
 - Theme
 - Guest account
 - Settings
-

Phase 2 — Authentication

- Google

- Apple
 - Email
 - Guest
 - Account linking
 - Logout
 - Delete account
-

Phase 3 — Reminder Engine

This is the most technically important phase.

- Reminder model
 - Recurrence engine
 - Local scheduler
 - Notifications
 - Alarm
 - Snooze
 - Completion
 - Missed reminders
 - Timezone
 - Reboot recovery
-

Phase 4 — Tasks

- Task UI
- Categories
- Priority
- Tags
- Recurring tasks
- Date range
- Progress

Phase 5 — Birthday/People

- People
- Birthday
- Anniversary
- Countdown
- Custom reminder schedule
- Birthday-day experience

Phase 6 — Calendar

- Month
- Week
- Day
- Agenda
- Filters

Phase 7 — Widgets

Android + iOS widgets.

Phase 8 — Cloud Sync

- Supabase
- Sync engine
- Backup
- Restore
- Multi-device

Phase 9 — Smart Features

- Voice

- Natural-language reminders
 - AI birthday messages
 - Smart recurrence
 - Templates
-

71. Testing Strategy

This app needs more than normal UI testing because **time is the core product**.

Test:

Time

23:59 → 00:00

Month change

Year change

Leap year

DST

Timezone change

Recurrence

Daily

Weekly

Monthly

Yearly

Custom

End date

No end date

Device

Restart

Force close

Battery saver

Offline

Notification disabled

Permission revoked

App updated

Birthday

5 days before

1 day before

Birthday day

After birthday

Next year

Leap-year birthday edge cases

72. QA Test Example

Your exact example becomes a mandatory automated test:

Person:

Father

Birthday:

30 October 2026

Start:

25 October 2026

Time:

22:00

Frequency:

Daily

Expected:

25 Oct 22:00 ✓

26 Oct 22:00 ✓

27 Oct 22:00 ✓

28 Oct 22:00 ✓

29 Oct 22:00 ✓

30 Oct birthday notification ✓

31 Oct

No notification ✓

Then:

30 Oct 2027

The yearly birthday rule should generate the next cycle.

73. App Store / Play Store Readiness

Before release:

Android

- Production signing
- App Bundle
- Notification permission
- Exact alarm policy review
- Privacy policy
- Data safety form
- Account deletion
- Google authentication
- Crash monitoring

iOS

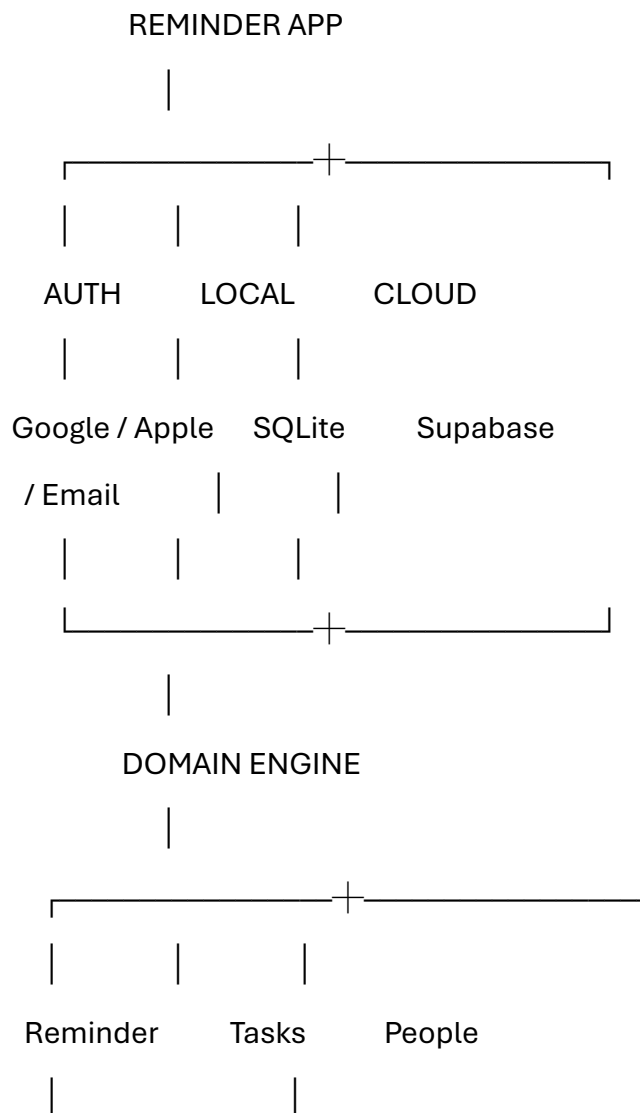
- App Store metadata
- Sign in with Apple

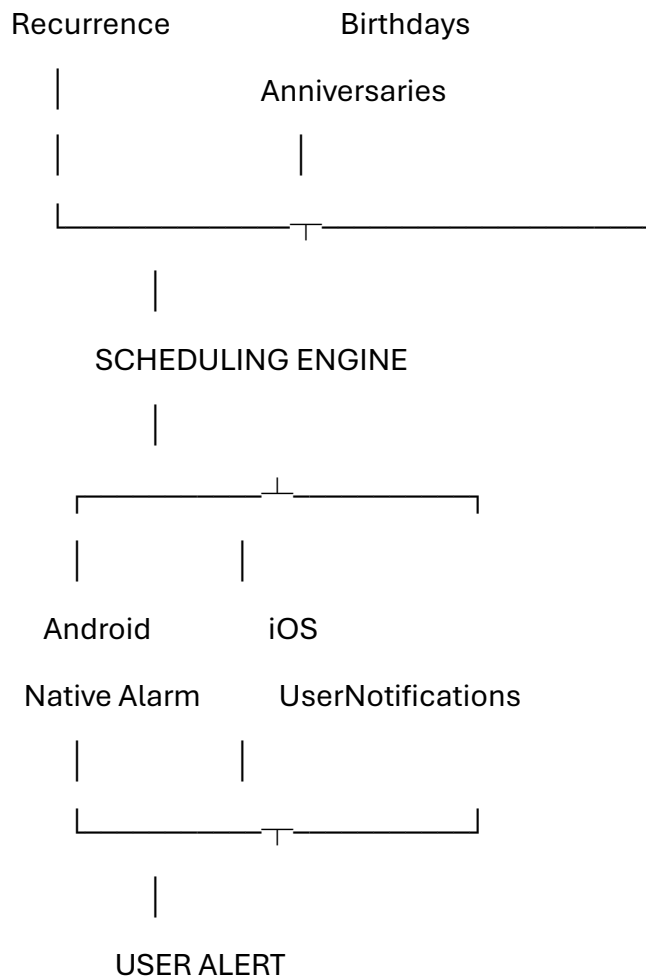
- Privacy nutrition labels
- Notification permissions
- Account deletion
- Privacy policy
- App Review testing account if needed

Apple specifically requires developers to account for privacy practices of third-party SDKs and privacy manifests where applicable, so the SDK stack should be kept lean. ([Apple Developer](#))

74. Final Product Architecture

The final system should look like this:





75. Final Screen Inventory

The first complete release should contain approximately:

Authentication — 8

1. Splash
2. Welcome
3. Login
4. Google account flow
5. Apple login
6. Email login
7. Profile setup
8. Permissions

Home — 6

- 9. Dashboard
- 10. Today
- 11. Upcoming
- 12. Next Reminder
- 13. Notification center
- 14. Search

Reminder — 9

- 15. Quick Add
- 16. Create Reminder
- 17. Advanced Reminder
- 18. Repeat
- 19. Custom Repeat
- 20. Alert Configuration
- 21. Alarm Configuration
- 22. Reminder Detail
- 23. Edit Reminder

Tasks — 6

- 24. Tasks
- 25. Task Detail
- 26. Create Task
- 27. Categories
- 28. Tags
- 29. Completed

People — 7

- 30. People
- 31. Person Detail
- 32. Add Person
- 33. Birthday

34. Birthday Detail

35. Anniversary

36. Important Date

Calendar — 4

37. Month

38. Week

39. Day

40. Agenda

Settings — 10+

41. Profile

42. Account

43. Appearance

44. Notifications

45. Sound

46. Widgets

47. Backup

48. Sync

49. Privacy

50. About

Plus:

- Empty states
- Error states
- Permission states
- Offline states
- Loading states
- Confirmation dialogs
- Bottom sheets
- Notification interactions

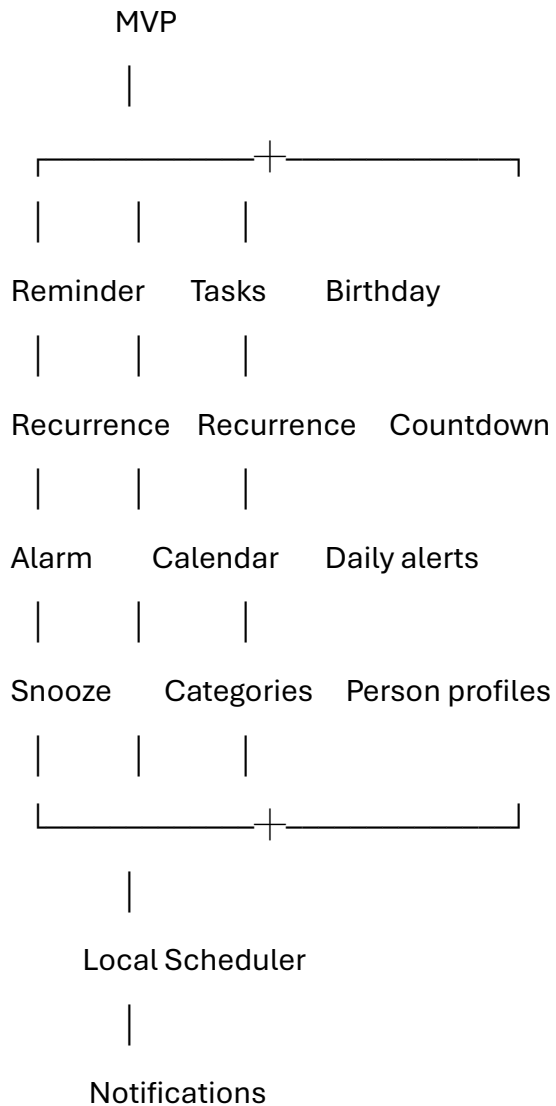
76. My Recommended MVP Boundary

Don't start by building AI.

The first production milestone should be:

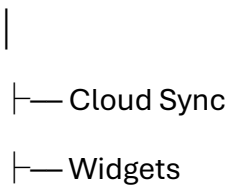
Beautiful UI + extremely reliable reminders.

Specifically:



Then:

V2



- └— Voice
 - └— Natural Language
 - └— Templates
 - └— Smart Features
-

77. One Important Change I'd Make to Your Original Idea

I would make **"Reminder Rules"** the central concept instead of having completely separate systems for Daily Tasks, Birthdays, Anniversaries and Events.

For example:

Reminder Rule

|

└— What?

| Father's Birthday

|

└— When?

| 30 October

|

└— Alert Window?

| 25–30 October

|

└— Frequency?

| Daily

|

└— Time?

| 10 PM

|

└— Special Event?

Birthday

Then the same engine can power:

"Remind me every day at 10 PM from 25 October until 30 October."

and:

"Remind me every Sunday at 8 PM."

and:

"Remind me 7 days before my insurance expires."

and:

"Remind me every month on the 15th."

That gives you a **single reliable scheduling architecture** instead of building four separate reminder systems.

Final design direction

The attached reference should be treated as the **visual inspiration**, with the finished product becoming:

Soft purple/lilac environment + glass cards + subtle tactile/skeuomorphic controls + compact typography + pill filters + floating navigation + beautiful illustrations + highly functional scheduling UI.

And technically:

Flutter + Clean Architecture + Riverpod + local SQLite/Drift + native Android/iOS scheduling + Supabase cloud sync + Google/Apple authentication.

For Google authentication, use the current supported Google Sign-In integration rather than an old custom OAuth flow; for Android, Google's current direction is Credential Manager rather than the legacy Google Sign-In API. ([Dart packages](#))

For iOS, build Apple sign-in into the authentication architecture from the beginning rather than adding it after the Android/Google flow is finished. ([Apple Developer](#))

This is now detailed enough to move from product planning into actual screen-by-screen implementation. The next development artifact should be the **complete UI specification for every screen**—exact component hierarchy, spacing, colors, typography, states, interactions, navigation routes, and Flutter widget/component names—so an AI coding agent can build the application consistently rather than improvising each screen.